

一、选题依据（包括目的、意义、国内外研究现状和发展趋势）

1.1 研究背景与目的

随着大数据时代图像和视频内容的爆炸式增长，高效、准确的内容检索技术已成为多媒体信息处理领域的关键。哈希（Hashing）作为一种将高维数据映射到低维二进制编码的技术，因其在存储和检索效率上的显著优势，成为大规模相似性检索的主流方法。其中，基于深度学习的哈希方法，如深度监督哈希（DSH）、DCPH 等，通过端到端学习，极大地提升了哈希码的表示能力和检索精度。

近年来，Central Similarity Quantization (CSQ) 等方法在保证哈希码量化误差最小化的同时，通过引入“中心相似性”概念，进一步提高了检索效率和性能，展现出巨大的应用潜力。这些哈希方法在电商推荐、版权保护、安全监控等领域扮演着日益重要的角色。

然而，现有多数深度哈希研究，包括 CSQ 在内，主要聚焦于提升检索性能和效率。在数据流的整个生命周期中，对于哈希处理过程引入的潜在安全漏洞及其防范关注不足。特别是在敏感数据处理和安全应用场景下，哈希码的生成、传输和存储可能面临数据篡改、信息泄露等风险。例如，未经授权地篡改原始输入数据可能导致错误的哈希结果，进而影响后续决策；而哈希码本身的暴露，即使其不直接包含原始数据，也可能通过模式分析泄露部分敏感信息。

本研究的目的是，在于探索并实践一套简单、高效且易于集成的密码学安全增强方案，以弥补现有深度哈希方法在数据完整性与信息机密性方面的不足。通过在核心哈希算法（以 CSQ 为例）外部构建安全防护层，我们旨在确保原始数据的处理完整性，并保护生成哈希码在传输与存储过程中的机密性，从而提升整个哈希检索系统的鲁棒性和可信赖度。

1.2 国内外研究现状与发展趋势

当前，国内外关于深度哈希的研究主要集中在以下几个方面：哈希性能优化，通过设计新型网络架构、损失函数或量化策略，提高哈希码的判别性和检索精度，减少量化误差，如[1]提出的 CSQ 方法。多模态哈希，针对图像-文本、视频-文本等多模态数据，研究如何学习统一的哈希空间以实现跨模态检索。小样本/零样本哈希，解决训练数据稀缺或类别未知情况下的哈希学习问题。

尽管深度哈希在性能上取得了显著进展，但其在安全性方面的考量相对滞后。少数研究开始关注深度哈希的对抗性攻击（Adversarial Attacks），即通过微小扰动改变原始数据，导致哈希模型生成错误的哈希码，从而规避检测或干

扰检索结果[2]。然而，这些研究多集中于攻击手段的分析与模型内部鲁棒性的提升，并未系统性地探讨数据完整性验证和哈希码机密性保护等基础设施层面的安全需求。

密码学领域在数据完整性验证（如消息认证码 MAC、数字签名）和数据机密性保护（如对称加密、非对称加密）方面已发展出成熟且经过严格验证的技术 [3]。这些技术广泛应用于通信安全、数据存储等领域。然而，如何将这些通用密码学原语，以最小的开销和最高的兼容性，集成到像 CSQ 这样的高性能哈希检索系统中，以解决特定安全需求，仍是一个值得深入探讨的方向。

未来的发展趋势预计将从单纯追求性能向兼顾性能与安全的方向演进。随着深度学习模型在更多关键应用中部署，对抗攻击、数据隐私和模型可靠性将成为不可忽视的挑战。将传统密码学安全机制与现代深度学习哈希方法相结合，构建“安全感知”的哈希检索系统，将是重要的发展方向，旨在为用户提供更可信、更安全的数据检索服务。

1.3 创新点

本研究的创新点在于：

提出并实践了一种轻量级、非侵入式的 CSQ 哈希检索系统安全增强方案。该方案在不修改 CSQ 核心算法的前提下，通过集成密码学安全哈希和对称加密技术，有效保障了原始数据的完整性及 CSQ 哈希码的机密性。强调并解决了深度哈希系统在实际部署中易被忽视的安全漏洞。与现有关注模型鲁棒性或对抗攻击防御的研究不同，本研究从系统层面考量，通过引入成熟密码学机制，防范数据篡改和哈希码信息泄露的风险。为未来“安全感知”的深度哈希系统设计提供了一个可行的基础框架。所提出的方案模块化且易于推广，可为其他深度哈希方法提供类似的安全性加强思路。

二、研究内容（具体研究内容，重点解决的问题，预期结果）

本研究在现有 CSQ（Central Similarity Quantization）方法的基础上，致力于解决其实际应用中可能存在的两大安全挑战：原始数据完整性验证和 CSQ 生成哈希码的机密性保护。为此，本研究将聚焦于提出并验证一个结合密码学安全机制的 CSQ 增强方案。

2.1 具体研究内容

本研究将围绕构建一个对原始数据进行完整性验证，并对 CSQ 生成的哈希码进行机密性保护的增强型哈希检索框架展开，具体研究内容包括

1) CSQ 模型集成与基准性能评估:

部署并复现《Central Similarity Quantization for Efficient Image and Video Retrieval》论文中提出的 CSQ 模型，并使用标准数据集（如 CIFAR-10, ImageNet 等，根据具体资源和需求选择）对其在相似性检索任务上的性能（例如，mAP、检索速度、哈希码生成速度）进行基准测试。这将作为后续安全增强方案引入后的性能对比基准。确保对原始 CSQ 方法的正确理解和实现，并为后续的安全增强方案提供可靠的性能基线。

2) 原始数据完整性保障机制设计与实现:

设计并实现一个基于 SHA-256 哈希算法的模块。该模块将在原始数据（例如图像、视频特征向量）输入 CSQ 模型之前，计算数据的 SHA-256 哈希值。该哈希值将与 CSQ 生成的哈希码一同存储或传输。确保原始数据在进入 CSQ 处理流程前以及在存储/传输过程中未被篡改，从而保证 CSQ 模型输入数据的可靠性。

3) CSQ 哈希码机密性保护机制设计与实现:

设计并实现一个基于 AES（Advanced Encryption Standard）对称加密算法的模块。该模块将在 CSQ 模型输出哈希码之后，对其进行加密处理。加密后的 CSQ 哈希码将替代原始 CSQ 哈希码进行存储或传输。同时，需要设计并实现相应的解密模块，以便在进行相似性检索前恢复原始 CSQ 哈希码。保护 CSQ 生成的哈希码在存储和传输过程中的机密性，防止未经授权的访问者获取和分析这些敏感的相似性信息。

4) 密钥管理策略的初步探讨与实现:

考虑到对称加密方案对密钥安全性的高度依赖，本研究将初步探讨并实现一套简化的密钥管理策略。这可能包括密钥的生成、安全存储（例如，使用环境变量或受保护的配置文件）、加载和更新机制。确保对称加密密钥本身的安全性，这是整个机密性保护方案有效性的基石。

5) 集成与端到端系统构建:

将上述各个模块（CSQ 模型、SHA-256 模块、AES 加密/解密模块、密钥管理模块）进行整合，构建一个端到端的安全增强型 CSQ 相似性检索系统。验证各模块之间的协同工作能力，形成一个完整的、可操作的解决方案。

6) 性能与安全性评估:

对整合后的系统进行全面的性能评估，包括 CSQ 哈希码生成时间、相似性检索时间、SHA-256 计算时间、AES 加密/解密时间等，并与纯 CSQ 基准性能进行对比，量化引入安全机制所带来的额外开销。同时，从理论和实践角度评估方案所能提供的安全级别（例如，对篡改的检测能力、对窃听的抵抗能力）。评估方案的实际可行性，并量化安全增强与性能之间的权衡。

2.2 重点解决的问题及对应方案

本研究将重点解决以下核心问题以及对应方案

- 1) 如何在不修改 CSQ 核心算法的前提下，为 CSQ 的输入数据提供密码学级别的完整性保障？引入 SHA-256 哈希对原始数据进行预处理和验证。
- 2) 如何确保 CSQ 模型生成的哈希码在存储和传输过程中的机密性，防止未经授权的暴露？引入 AES 对称加密对 CSQ 哈希码进行后处理和保护。
- 3) 所引入的密码学安全机制对 CSQ 相似性检索的效率和准确性有何影响？通过详细的性能评估，量化额外计算开销，并确认对相似性检索结果无负面影响。
- 4) 如何设计一个基础的、易于实施的密钥管理方案，以确保对称加密的有效性？探讨并实现简化的密钥生成、存储和加载机制。

2.3 预期结果

本研究预期达到以下成果：

- 1) 一个功能完善的安全增强型 CSQ 相似性检索系统原型： 该系统将能够接收原始数据，为其生成密码学安全的完整性哈希，通过 CSQ 模型生成相似性哈希码，并对这些哈希码进行机密性保护，最终实现安全的相似性检索。
- 2) 详细的性能评估报告： 报告将量化在 CSQ 系统中引入 SHA-256 完整性验证和 AES 哈希码机密性保护所带来的额外计算开销，并与原始 CSQ 基准性能进行对比。预期增加的开销在可接受范围内，且不影响 CSQ 的核心检索效率。
- 3) 安全性分析和评估： 明确所提出方案能够提供的安全保障范围，例如能够有效检测数据篡改、抵抗未经授权的哈希码窃听等，并指出方案的局限性。
- 4) 可复用的代码模块： 形成一套模块化的代码库，包括 SHA-256 模块、AES

加密/解密模块和 CSQ 模型集成部分，便于后续研究和实际应用。

通过上述研究内容和预期结果，本研究旨在为 CSQ 这类高效哈希检索方法在实际安全敏感环境中的应用提供一个可靠且易于实施的安全增强框架。

三、研究方案（包括研究方法、技术路线、可行性分析等）

本研究旨在通过集成密码学安全机制，构建一个安全增强型 CSQ 相似性检索系统。本部分将详细阐述为实现这一目标所采用的研究方法、具体技术路线，并对研究的可行性进行分析。

3.1 研究方法

本研究将采用理论分析与实验验证相结合的方法：

3.1.1 文献研究法： 深入研究深度哈希、CSQ 模型、密码学安全哈希函数（如 SHA-256）和对称加密算法（如 AES）的最新进展和理论基础，为本研究的设计提供坚实的理论支撑。

3.1.2 系统设计与模块化开发： 采用模块化设计思想，将系统拆分为 CSQ 模型、完整性验证模块、机密性保护模块和密钥管理模块等，明确各模块的功能、接口和交互方式，便于独立开发、测试和集成。

3.1.3 实验对比分析法： 在标准数据集上进行实验，通过定量指标（如 mAP、查询时间、哈希生成时间、加密解密时间等）评估引入安全机制前后系统的性能变化，并进行详细的对比分析。

3.1.4 安全评估法： 从理论上分析所引入密码学机制能够提供的安全级别，并尝试通过模拟攻击场景（如数据篡改、哈希码窃取）验证系统的安全性。

3.2 技术路线

本研究的技术路线遵循自底向上、模块化集成的原则，主要分为以下几个阶段：

3.2.1 阶段一：CSQ 基准系统搭建与性能评估

复现 CSQ 模型，基于 PyTorch 或 TensorFlow 等深度学习框架，实现或调用 CSQ 模型。数据集准备，选择合适的公开数据集（如 CIFAR-10、ImageNet 等）进行数据预处理。基准性能测试，在选定数据集上，评估原始 CSQ 模型

在哈希码生成速度、检索精度（mAP）和检索速度等方面的性能指标，作为后续对比的基准。产出可运行的 CSQ 模型代码，详细的基准性能测试报告。

3.1.2 阶段二：原始数据完整性验证模块设计与实现

SHA-256 哈希模块实现，开发一个独立的 Python 模块，封装 SHA-256 算法，能够接收任意输入数据（例如图像文件、特征向量等），并输出其对应的 SHA-256 哈希值。集成到 CSQ 数据预处理流程：将 SHA-256 模块集成到 CSQ 模型的数据输入端。在数据进入 CSQ 模型前，计算其 SHA-256 值并保存。产出可复用的 SHA-256 哈希模块，集成完整性验证功能的 CSQ 数据输入流程。

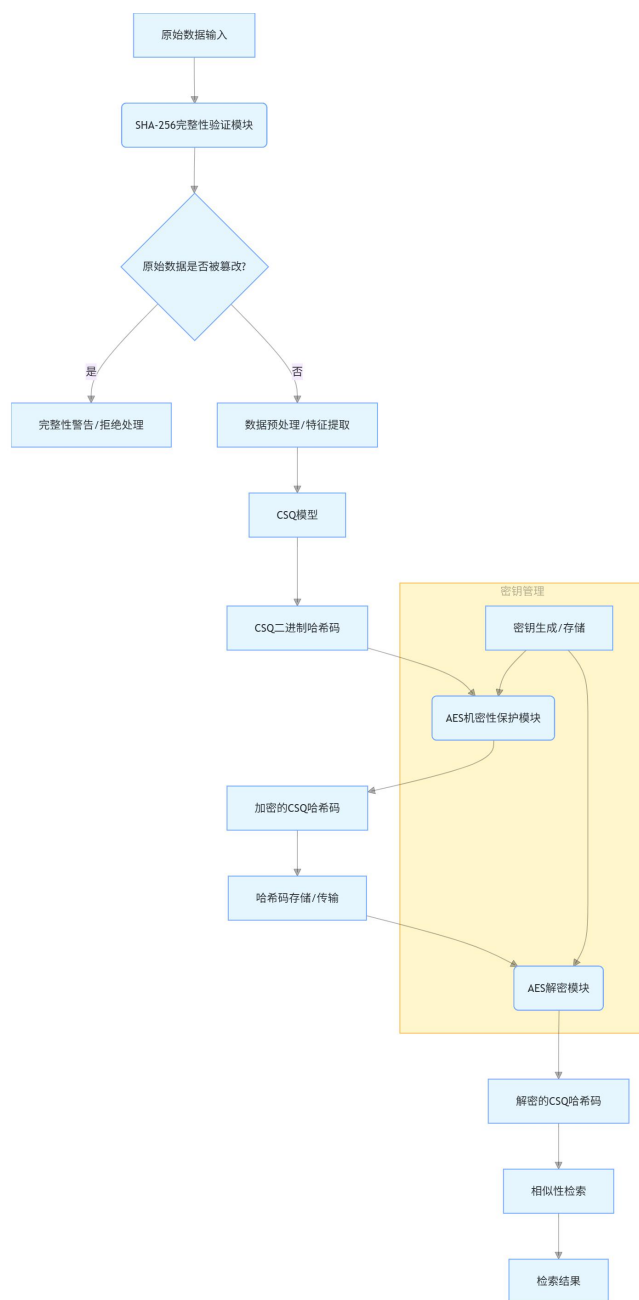
3.2.3 阶段三：CSQ 哈希码机密性保护模块设计与实现

AES 加密/解密模块实现，开发一个独立的 Python 模块，封装 AES 对称加密算法（选择合适的模式，如 AES-256-CBC 或 GCM），实现对字节流的加密和解密功能。密钥管理策略实现，设计并实现密钥生成、存储和加载的简易机制（例如，从环境变量、配置文件安全加载）。集成到 CSQ 哈希码输出流程，将加密模块集成到 CSQ 模型输出哈希码的后端，对生成的二进制哈希码进行加密处理后再进行存储或传输。集成到检索流程，在进行相似性检索前，对加密的哈希码进行解密，恢复原始 CSQ 哈希码，然后进行相似性计算。产出可复用的 AES 加密/解密模块，简易密钥管理模块，集成机密性保护功能的 CSQ 哈希码存储/检索流程。

3.2.4 阶段四：系统集成、性能与安全性评估

系统整合，将所有独立开发的模块进行整合，构建一个端到端的安全增强型 CSQ 相似性检索系统原型。性能评估，对比基准 CSQ 模型，量化安全增强系统在哈希码生成时间、检索时间上的额外开销。评估 SHA-256 计算时间和 AES 加密/解密时间的性能瓶颈。分析安全机制对检索精度（mAP）的影响（理论上不应有影响）。安全性验证，模拟数据篡改，在完整性验证模块处理前篡改原始数据，验证系统是否能正确检测出。模拟哈希码窃取，在不解密的情况下尝试分析加密哈希码，验证机密性保护的有效性。产出完整的安全增强型 CSQ 系统原型，详细的性能与安全性评估报告。

技术路线图示：



3.3 可行性分析

本研究在技术、资源和时间上均具备可行性。

3.3.1.技术可行性

深度哈希与 CSQ 模型，深度哈希作为当前热点研究领域，其理论和实现方法已相对成熟。CSQ 模型作为代表性的方法，已有公开的论文和可能的参考实现，复现和集成难度适中。密码学安全算法，SHA-256 和 AES 是工业界

和学术界广泛接受和使用的标准密码学算法，各种编程语言（如 Python）均提供了成熟、高效且经过验证的实现库，可以直接调用。这大大降低了从零开始实现算法的难度。模块化集成，将安全性功能作为独立模块进行封装，在 CSQ 模型的外围构建安全层，避免了对 CSQ 模型内部逻辑的复杂修改，降低了集成风险。

3.3.2.资源可行性

软硬件环境，本研究所需的深度学习框架（PyTorch/TensorFlow）、编程语言（Python）及其相关库（如 OpenCV、Numpy、Scipy、PyCryptodome 等）均为开源且易于获取。实验所需的计算资源（GPU、CPU）在实验室或云平台上均可获得。数据集，图像相似性检索领域有大量公开的标准数据集（如 CIFAR-10、ImageNet、NUS-WIDE 等），可以满足实验需求。

3.3.3.时间可行性

本研究设定了清晰的研究目标和分阶段的技术路线，每个阶段的任务分解明确。在合理的时间规划下，预期能够按时完成各项研究任务。特别是对成熟密码学库的直接调用，将大大缩短开发周期。

综上所述，本研究的选题具有重要意义，研究内容明确，技术路线清晰合理，且在技术、资源和时间上均具备较强的可行性。

四、研究计划与进度安排

第一阶段：前期准备与 CSQ 基准系统搭建。

- 1. 完成 CSQ 相关文献综述
- 2. 搭建开发环境，熟悉相关库与框架
- 3. 复现 CSQ 模型并进行基准性能测试

第二阶段：安全模块设计与实现

- 1. 设计并实现 SHA-256 完整性验证模块
- 2. 设计并实现 AES 加密/解密模块
- 3. 初步设计并实现密钥管理机制

第三阶段：系统集成与初步测试

- 1. 将安全模块与 CSQ 系统进行集成

2. 进行系统功能测试和初步性能评估

第四阶段：性能与安全性评估

1. 进行详细的性能对比测试（时间、mAP）
2. 进行安全性测试与分析（完整性、机密性）

五、工作基础及条件

处理器，12th Gen Intel(R) Core(TM) i5-12500H (2.50 GHz)
显卡，Intel(R) Iris(R) Xe Graphics
系统类型，64 位操作系统，基于 x64 的处理器
机带 RAM，16.0GB

六、参考文献目录

- [1] R. Yang, W. Zhang, S. Zhang, W. J. Song, and Q. Liu, "Central Similarity Quantization for Efficient Image and Video Retrieval," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021, pp. 11099-11108. (Original paper, for background context only)
- [2] Y. Lin, C. Chen, X. Chen, and B. Jin, "Adversarial Hashing for Image Retrieval," in Proceedings of the 26th ACM International Conference on Multimedia (MM '18), 2018, pp. 138-146.
- [3] W. Stallings, Cryptography and Network Security: Principles and Practice, 8th ed. Pearson, 2020.
- [4] 张志升. 基于深度监督哈希的高效图像检索研究[D]. 山东:山东建筑大学,2023.
- [5] Li Yuan, Tao Wang, Xiaopeng Zhang, Francis EH Tay, Zequn Jie, Jiashi Feng. *Central Similarity Quantization for Efficient Image and Video Retrieval*. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020.

